Caching in Scala using Cats Ref

Katrina Petrevice

What is Ref?

A concurrent mutable reference: safe, pure, and composable.



Mutable

The value changes over time. Updates are atomic, with no locks.

Concurrent

Many fibres can read and write it at once, safely.

Pure

Creating and using a Ref are IO values. Nothing runs until the program does.

When to choose in-memory caching

Fast access

No network, no serialisation. Microsecond latency in JVM memory.

Hot data

Frequently accessed data is cheap to keep in memory.

Small, bounded data

Config and lookup tables. A few thousand entries keep an unbounded Map safe.

Independent instances

Single or independent instances. Duplication across servers is acceptable.

When to choose cloud caching

Shared state

Multiple servers need a consistent cache.

Persistence

The cache survives restarts, with a TTL.

Larger capacity

Store datasets larger than the heap.

Global invalidation

Update once, every instance sees it on the next read.

Trade-offs when implementing caching

1	**Cold start**	An empty cache on startup.
2	**Low hit rate**	Keys too granular to be reused.
3	**Invalidation**	Stale data versus event-driven updates.
4	**Memory overhead**	Unbounded growth and infrastructure cost.

Cold start

Trade-off 1 of 4

Problem

The cache is empty on startup, so early requests are slow.

Why it happens: there is no historical data to serve on initialisation.

Fixability: inherent but mitigatable

Warm up before traffic

Pre-load top data before the readiness probe passes.

Compute each key once

Single flight: a cold burst triggers one load.

Accept some latency

The first few minutes may be slower.

Low cache hit rate

Trade-off 2 of 4

Problem

Many unique data instances reduce cache reuse.

Why it happens: cache keys are too granular.

Fixability: yes, highly fixable

Composite keys

user + category, not user + timestamp.

Normalise keys

Trim, lowercase and sort parameters.

Measure first

Count hits and misses before tuning.

Cache invalidation

Trade-off 3 of 4

Challenge

Choosing between stale-tolerant (TTL) and event-driven invalidation.

Why it's tricky: no one-size-fits-all answer. It depends on freshness requirements.

Fixability: yes, with the right strategy

Time-based (TTL)

Expires after 5 minutes. Add jitter to avoid herds.

Event-driven

Drop the key on change: update(_ - key).

Stale-while-revalidate

Serve the old value while one fibre refreshes.

Memory and cost overhead

Trade-off 4 of 4

Problem

RAM is expensive and limited. Redis and CDNs add operational cost.

Why it happens: unbounded cache growth plus expensive infrastructure.

Fixability: yes, controllable

LRU size limits

Evict the Least Recently Used entries.

Monitor utilisation

Alert on size and hit-rate thresholds.

Evict expired entries

Our cache doesn't yet. We fix it later.

Scenario: API request and response

Use case

Caching on the Service layer

Goal

User wants an immediate response on the UI page

For example, p95 under 100 ms

Request

The key comes from the request id.

→

Look up in Ref

Is there an entry younger than the TTL?

→

Hit or miss

Hit: return it.
Miss: compute and store.

→

Response

Fast on a hit, correct on a miss.

The Naive Approach

Naive Approach: global Ref

Production code

CacheService.scala

```scala
object CacheService {
  private val ttl = Duration.ofMinutes(5)

  val cacheRef =
    Ref.unsafe[IO, Map[String, (Response, Instant)]](Map.empty)
}
```

Key and value: the request id maps to the response and when it was cached.

Ref.unsafe: allocated outside IO, so every test shares this one cache.

Naive Approach: reading

Production code

CacheService.scala

```scala
def cacheResponse(request: Request): IO[Response] =
  for {
    now    <- IO(Instant.now())
    cached <- cacheRef.get.map(_.get(request.id))
    response <- cached match {
      case Some((resp, at)) if isValid(at, now) => IO.pure(resp)
      case _ => updateCache(request)
    }
  } yield response
```

Two separate reads: the clock via IO(Instant.now()), the cache via get.

Hit or miss: a fresh entry is returned, otherwise it is recomputed.

Naive Approach: writing

Production code

CacheService.scala

```scala
def updateCache(request: Request): IO[Response] =
  for {
    now      <- IO(Instant.now())
    response <- computeResponse(request)
    entry = (response, now)
    _        <- cacheRef.update(_.updated(request.id, entry))
  } yield response
```

A second step: this write is separate from the read on the previous slide.

The gap: another fibre can run between the read and the write. That is the race.

Testing the Naive Approach

Test: expected behaviour

Test code

NaiveExpectedSuite.scala

```scala
test("returns the cached response within the TTL") {
  for {
    first  <- CacheService.cacheResponse(Request("someId"))
    second <- CacheService.cacheResponse(Request("someId"))
  } yield assert(first == second)
}
```

First call: computes and caches.

Second call: inside the TTL, so served from the cache.

Equal means cached: nanoTime makes each computation differ.

Output: expected behaviour

Terminal output

sbt (real output, trimmed)

```
$ sbt testOnly naive.NaiveExpectedSuite
naive.NaiveExpectedSuite:
  + returns the cached response within the TTL 0.162s
[info] Passed: Total 1, Failed 0, Errors 0, Passed 1
[success] Total time: 1 s, completed 3 Oct 2026, 19:38:16
```

Pass: the first and second responses are equal.

Fast: 0.162s, with nothing to wait for.

Why is CacheService tricky to test?

Time-dependent behaviour

Challenge 1 of 3

Problem

You can't use the real `Instant.now()`. Tests become slow and flaky.

Solution

Inject the clock, so the test controls time instead of waiting for it.

A clock the test owns can jump forward in one step: no sleep, no waiting.

Pitfall 1: Time

Flaky Test code

NaiveTimeSuite.scala

```scala
test("respects the TTL (FLAKY)") {
  for {
    first  <- CacheService.cacheResponse(Request("someId"))
    _      <- IO.sleep(1.minute)
    second <- CacheService.cacheResponse(Request("someId"))
  } yield assert(first != second)
}
```

Really waits: a full real minute.

Wrong either way: longer is slow, shorter is wrong.

Edge cases: exactly at the TTL is nearly untestable.

Output: slow and failing

Terminal output

sbt (real output, trimmed)

```
$ sbt testOnly naive.NaiveTimeSuite
naive.NaiveTimeSuite:
==> X …TimeSuite.respects the TTL (FLAKY)  60.179s
munit.FailException: …/NaiveTimeSuite.scala:18 assertion failed
17:       second <- CacheService.cacheResponse(Request("someId"))
18:     } yield assert(first != second)
[error] Failed: Total 1, Failed 1, Errors 0, Passed 0
```

Slow: 60 real seconds for a single test.

Fails: a 1-minute sleep is inside the 5-minute TTL.

To pass: it must really wait longer than 5 minutes.

Shared mutable state

Challenge 2 of 3

Problem

`Ref.unsafe` on a global object causes state to bleed between tests.

Issue

Tests interfere. Resetting isn't reliable when suites run in parallel.

Fix: allocate a fresh Ref per test with Ref.of, not one at class-load time.

Pitfall 2: State bleed

State bleed | Test code

NaiveStateBleedSuite.scala

```
test("populates the cache") {
  CacheService.cacheResponse(Request("key1"))
    .map(r => assert(r.value.nonEmpty))
}
private val msg = "Cache should start empty!"
test("starts with an empty cache") {
  CacheService.cacheRef.get.map(c => assert(c.isEmpty, msg))
}
```

Order dependent: passes alone, fails if the first test ran before it.

One shared Ref: allocated at class-load time, so state bleeds.

Output: state bleed (alone)

Terminal output

sbt (real output, trimmed)

```
$ sbt testOnly naive.NaiveStateBleedSuite -- --tests=.*empty.*
==> i …StateBleedSuite.populates the cache ignored 0.0s
naive.NaiveStateBleedSuite:
  + starts with an empty cache 0.173s
[info] Passed: Total 1, Failed 0, Errors 0, Passed 1, Ignored 1
```

Passes alone: the Ref starts empty, so the assertion holds.

Other test skipped: the filter ignored populates the cache.

Output: state bleed (suite)

Terminal output

sbt (real output, trimmed)

```
$ sbt testOnly naive.NaiveStateBleedSuite
naive.NaiveStateBleedSuite:
  + populates the cache 0.155s
==> X …StateBleedSuite.starts with an empty cache  0.01s
munit.FailException: …/NaiveStateBleedSuite.scala:14
Cache should start empty!
[error] Failed: Total 2, Failed 1, Errors 0, Passed 1
```

Passes: **populating the cache runs first and fills the Ref.**

Fails: **the next test starts with that cache still full.**

Concurrent edge cases

Challenge 3 of 3

Race condition

The cache read and the cache update are two separate steps, not one atomic step.

Consequence

A cache stampede: many callers miss at once and all hit the backend.

Each Ref operation is atomic. A get followed by an update is two operations, and atomic parts don't make an atomic whole.

Pitfall 3: Concurrency

Race Test code

NaiveConcurrentMissesSuite.scala

```scala
test("concurrent misses compute once") {
  val call = CacheService.cacheResponse(req)
  for {
    pair <- (call, call).parTupled
  } yield assert(pair._1 == pair._2)
}
```

Parallel calls: the same call runs twice at once, for one key.

Same test: we run it again on the refactored version later.

If the cache works: both responses are equal.

Output: fails on the naive cache

Terminal output

sbt (real output, trimmed)

```
$ sbt testOnly naive.NaiveConcurrentMissesSuite
naive.NaiveConcurrentMissesSuite:
==> X …MissesSuite.concurrent misses compute once  0.291s
munit.FailException: …/NaiveConcurrentMissesSuite.scala:15
assertion failed
14:       pair <- (call, call).parTupled
15:     } yield assert(pair._1 == pair._2)
[error] Failed: Total 1, Failed 1, Errors 0, Passed 0
```

Fails: the two responses are not equal.

Computed twice: each call missed and computed its own response.

Same test: we run it again on the refactored cache.

The Refactored Approach

Refactored: inject dependencies

Fixes Pitfalls 1, 2

Production code

CacheService.scala

```scala
object CacheService {
  def create(
    ttl: FiniteDuration,
    clock: Clock[IO] = Clock[IO]
  ): IO[CacheService] =
    Ref.of[IO, Map[String, CacheEntry]](Map.empty)
      .map(new CacheService(_, clock, ttl))
}
```

Companion object: it can call the class's private constructor.

Passed in: the TTL and the Clock are parameters, so a test controls both.

Ref.of: a fresh Ref on every call.

Refactored: memoize the work

Fixes Pitfalls 1, 3

Production code

CacheService.scala

```
for {
  now  <- clock.monotonic
  memo <- compute(req).memoize
  // ...
} yield response
```

Why memoize: an IO is a recipe: running it twice does the work twice.

memoize: runs the IO at most once and shares its result.

Monotonic clock: never jumps backwards.

Refactored: using modify

Fixes Pitfall 3

Production code

CacheService.scala

```
result <- cache.modify { entries =>
  entries.get(req.id) match {
    case Some(e) if now - e.cachedAt < ttl =>
      (entries, e.result)
    case _ =>
      (entries.updated(req.id, CacheEntry(memo, now)), memo)
  }
}
```

One step: decide and write together, nothing slips in between.

Hit: return the stored memoized IO.

Miss: store the new memoized IO and return it.

Key improvements

Clock passed in	**A test supplies its own clock.**
Ref.of, not Ref.unsafe	**Every service gets its own Ref, so tests never share state.**
memoize for single flight	**Each key is computed once and shared by concurrent callers.**
modify, not get then update	**Reads, decides and writes in one step, and returns a result.**

Testing the Refactored Approach

Test: expiry without sleeping

Fast | Test code

RefactoredTimeSuite.scala

```scala
test("expires entries after the TTL") {
  for {
    time    <- Ref.of[IO, FiniteDuration](0.seconds)
    service <- CacheService.create(5.minutes, fakeClock(time))
    first   <- service.cacheResponse(req)
    _       <- time.update(_ + 6.minutes)
    fresh   <- service.cacheResponse(req)
  } yield assert(first != fresh)
}
```

fakeClock: a Clock the test controls; it reports the time the Ref holds.

Time is a value: time.update moves it forward six minutes in one step.

No waiting: the test never sleeps; nothing outside it is involved.

Test: concurrent misses

Deterministic

Test code

RefactoredConcurrentSuite.scala

```scala
test("concurrent misses compute once") {
  for {
    service <- CacheService.create(5.minutes)
    call = service.cacheResponse(req)
    pair <- (call, call).parTupled
  } yield assert(pair._1 == pair._2)
}
```

Parallel calls: the same call runs twice at once, for one key.

Deterministic: whoever reaches modify first stores the memoized IO; the other reuses it.

Taking It Further

Evict expired entries

Production code

CacheService.scala

```scala
// inside CacheService: drop entries past their TTL
def evictExpired: IO[Unit] =
  clock.monotonic.flatMap { now =>
    cache.update(_.filter { case (_, e) =>
      now - e.cachedAt < ttl
    })
  }
```

One atomic update: filters the Map in a single cache.update.

Same rule as a read: kept only while now - cachedAt is under the TTL.

Sweep in the background

Production code

CacheService.scala

```scala
// run it in the background for the life of the app
def sweep(service: CacheService): Resource[IO, Unit] =
  (IO.sleep(1.minute) *> service.evictExpired)
    .foreverM
    .background
    .void
```

foreverM: repeats sleep then evict for as long as the app runs.

background: gives a Resource, so the loop is cancelled on shutdown.

Why Ref matters

Atomic shared state, without locks.

IN OUR CACHE

No locks

One modify replaced the race.

Testable

A fresh Ref for every test.

Computed once

Concurrent callers share a result.

BEYOND CACHING

Counters	Metrics and hit rates
Unique IDs	Sequence numbers
Rate limiters	Allow or reject a request
Config and flags	Read often, rarely changed

Summary

Ref

```
Ref[F[_], A]
Ref.of[F, A](a: A): F[Ref[F, A]]
```

A concurrent mutable reference that always holds a value.

modify

```
modify[B](f: A => (A, B)): F[B]
```

Reads, writes and returns a result in one atomic step. Our cache: decide hit or miss.

update

```
update(f: A => A): F[Unit]
```

Atomic write, no result. For when the new value depends only on the old: counters, removing a key.

Thank you

Questions?

Resources and references

Cats Effect: Ref (3.x)

typelevel.org/cats-effect/docs/std/ref

Cats Effect: Ref (2.x)

typelevel.org/cats-effect/docs/2.x/concurrency/ref